AUTOMATIC MELODY REDUCTION VIA SHORTEST PATH FINDING

Ziyu Wang[12] **Yuxuan Wu**[1] **Roger B. Dannenberg**[3] **Gus Xia**[1]
[1] Music X Lab, MBZUAI [2] New York University [3] Carnegie Mellon University
{ziyu.wang, yuxuan.wu, gus.xia}@mbzuai.ac.ae, rbd@cs.cmu.edu

ABSTRACT

Melody reduction, as an abstract representation of musical compositions, serves not only as a tool for music analysis but also as an intermediate representation for structured music generation. Prior computational theories, such as the Generative Theory of Tonal Music, provide insightful interpretations of music, but they are not fully automatic and usually limited to the classical genre. In this paper, we propose a novel and conceptually simple computational method for melody reduction using a graph-based representation inspired by principles from computational music theories, where the reduction process is formulated as finding the shortest path. We evaluate our algorithm on pop, folk, and classical genres, and experimental results show that the algorithm produces melody reductions that are more faithful to the original melody and more musically coherent than other common melody downsampling methods. As a downstream task, we use melody reductions to generate symbolic music variations. Experiments show that our method achieves higher quality than state-of-the-art style transfer methods. [1]

1. INTRODUCTION

Maintaining structural coherence in long-term music generation is a fundamental challenge. One approach to addressing this challenge is through hierarchical models, which rely on extracting high-level *abstractions* to enable cascaded generative processes [1–4]. These abstractions provide a coarser-grained view of musical structure, capturing essential long-range dependencies. In existing approaches, abstractions are typically explicitly defined (e.g., chord progression or phrase labels) or learned through unsupervised methods (e.g., latent codes via an autoencoder). Yet, so far, they have not been able to capture a fundamental musical structure: the *melodic flow*—how a melody evolves and resolves within a phrase—which remains too nuanced to be explicitly labeled and too challenging for unsupervised learning to reliably identify.

From a musicology perspective, melodic flow can be represented through *melody reduction*, which preserves the structural essence of a melody [5, 6]. However, most existing approaches regard melody reduction as a by-product of analysis, typically represented by hierarchical structures such as trees for further interpretation [7,8]. In this context, reduction is not a fixed transformation but rather a subjective and demonstrative projection of the analysis procedure. This inherent ambiguity makes melody reduction not only difficult to evaluate but also challenging to use as a practical representation [9–11]. In this work, we explore how melody reduction can be approximated using structural heuristics, aiming to make the concept more accessible and useful for music generation.

To this end, we propose an algorithm for automatic melody reduction. The algorithm uses the graph representation of a melody phrase and regards all possible reductions as graph paths. The intuition behind the algorithm is that if we define a cost function consistent with guiding principles underlying most reduction theories, an ideal reduction should be the path with the least cost. Specifically, we consider two principles. First, the subsequent notes in an ideal melody reduction usually reveal a simpler structure (e.g., a prolongation (unison), or a linear progression (step-wise motion) [5]. Second, an ideal melody reduction usually includes notes of higher significance in terms of pitch, rhythm, and harmony [6, 12]. We define edge costs based on these principles and use the shortest-path algorithm to find the melody reduction [13]. The resulting path is subsequently post-processed into an actual melody.

We evaluate the proposed algorithm in pop, folk, and classical music genres, showing that it yields reductions that are often perceived as more faithful to the original melody and musically coherent compared to other melody downsampling methods. We also introduce variation generation as a downstream application, in which we train a melody generation model conditioned on reductions. The reductions extracted with our algorithm are shown to yield higher-quality variations compared to baselines.

2. RELATED WORK

In this section, we review three realms of related work: 1) cognitive theories about music reduction, 2) the algorithmic implementation of music theories, and 3) the importance of melody reduction in downstream applications.

In the history of cognitive music theory, a shared

[1] Music samples of melody reduction and variation can be found at https://auto-melody-reduction.github.io/AMRA-demo/. We release the code at https://github.com/ZZWaang/melody-reduction-algo.

© Z. Wang, Y. Wu, R. Dannenberg, and G. Xia. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** Z. Wang, Y. Wu, R. Dannenberg, and G. Xia, "Automatic Melody Reduction via Shortest Path Finding", in *Proc. of the 26th Int. Society for Music Information Retrieval Conf.,* Daejeon, South Korea, 2025.

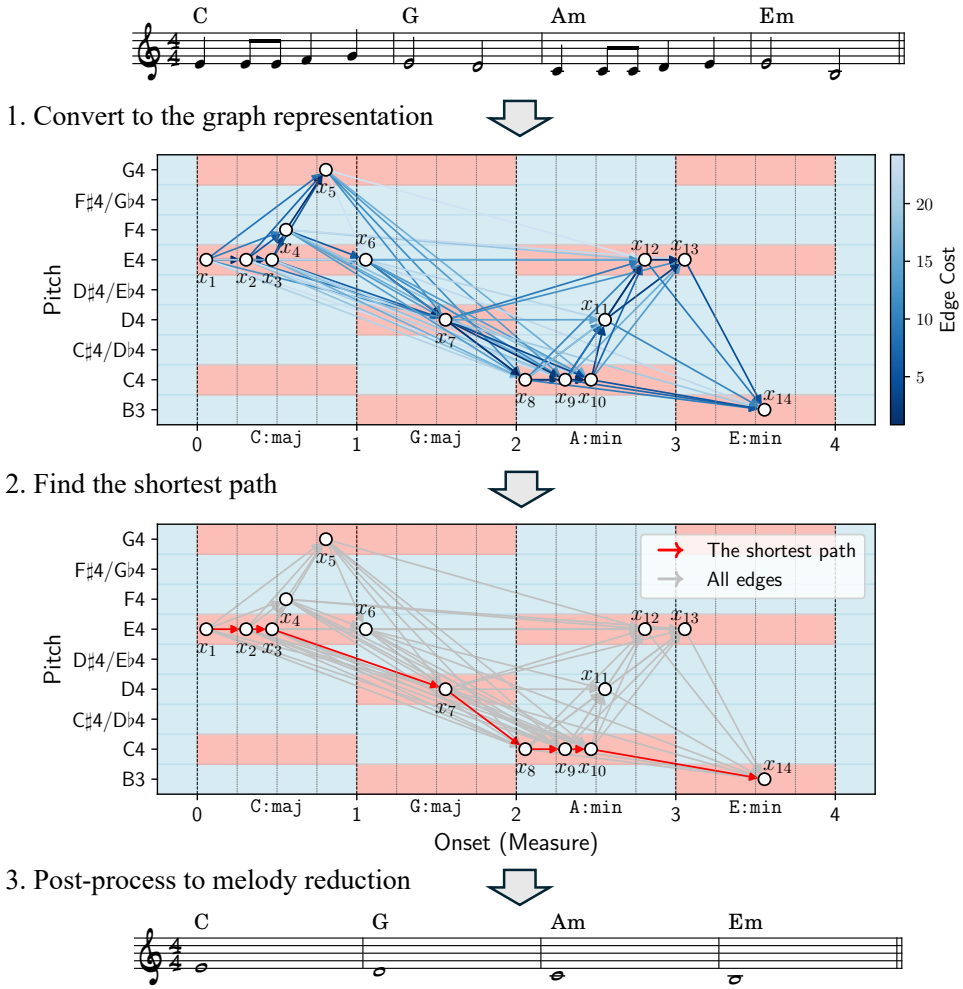


Figure 1: The overview of the proposed melody reduction algorithm.

methodology of music analysis is to use a reduced melody to represent the abstract melodic flow [5, 6, 14]. Schenkerian analysis involves a recursive reduction process to turn a music composition into the fundamental structure [5]; and the Generative Theory of Tonal Music (GTTM) further formalizes the grammar in Schenkerian analysis [6]. These studies highlight that melody reduction is an effective representation in the cognition process, and reduction is highly related to the considerations of note connection, harmonic context, pitch importance, etc., which usually imply tension and relaxation in different music scopes.

There are several attempts to turn these theories into algorithms. Kirlin *et al.* propose a framework for automatic Schenkerian analysis [15], and Hamanaka *et al.* design an interactive software to implement GTTM using machine-learning techniques [9, 16, 17]. Other approaches reduce melodies recursively by assigning weights to notes [7, 8]. However, a quality gap remains between automatic analyses and human interpretations. Moreover, the algorithms usually require score-notation level data (e.g., MusicXML format), are genre-specific, and are not open-sourced. A recent computational music analysis points out that since our music preference is hard to express in formal grammar, such formal systems tend to have a broad search space of music analyses [18]. This motivates us to pursue an intuitive alternative: we directly approximate melody reduction based on cognitive preference without building a formal system.

Although melody reduction is mostly studied in an analytical scope, recent advances in deep learning also show that melody reduction representation is beneficial for structured long-term music generation. Previously, melody reduction was usually implicitly modeled by surrogate features such as down-sampled melody statistics [1], melody contour [19], or implicit latent representations [20]. The recent hierarchical music generation methodology shows that using an explicitly defined melody reduction, long-term music generation can be tackled more elegantly and effectively [3]. The algorithm we propose aims to establish a foundation for such future studies.

3. METHODOLOGY

In this section, we introduce the proposed melody reduction algorithm in detail. A diagram of our algorithm is shown in Figure 1. Section 3.1 introduces the data attributes and the graph representation of a melody. Section 3.2 defines the edge types of the graph, and Section 3.3 defines the edge cost. Finally, we discuss the melody reduction post-processing operations in Section 3.4.

3.1 Graph Representation of a Melody

The input to the algorithm is a sequence of notes, denoted by $x_1, ..., x_N$, and an underlying chord progression, denoted by $c_1, ..., c_K$. A melody can be represented by a directed graph $\mathcal{G}(V, E)$, where the melody notes are regarded as graph nodes $V := \{x_i\}_{i=1}^{N}$, and temporal relations of notes can be represented by edges $E := \{x_i \to x_{i+1}\}_{i=1}^{N-1}$.

We consider the onset, pitch, and duration attributes of a note x_i. These are denoted by $\text{Onset}(x_i), \text{Pitch}(x_i)$ and $\text{Dur}(x_i)$, respectively. The onset and duration should be quantized by beat locations, and the pitch is represented by MIDI note numbers from 0 to 127. Additionally, a chord is represented by a 12-d binary chroma vector, i.e., $c_i \in \{0, 1\}^{12}$, and we define $\text{Chord}(x_i) \in \{c_1, ..., c_K\}$ to indicate chord membership of the note x_i. In our algorithm, we heuristically detect anticipation-like cases (a type of non-chord tone) and regard these notes as belonging to the next chord.

3.2 Edge Definition

In the proposed algorithm, we regard both the original melody and reduced melodies as paths from x_1 to x_N. The original melody uses edges in E, whereas a reduction uses shortcut edges. To this end, we define an *augmented* edge set $E^* := \{x_i \to x_j | i < j\}$, representing all causal edges. If an edge $x_i \to x_j$ is selected in the reduction process, it means the melodic movement from $x_i \to x_j$ is more significant than all other movements $x_{i'} \to x_{j'}$ inside the time range, i.e., $i \le i' < j' \le j$ and $(i, j) \ne (i', j')$.

We categorize an edge $x_i \to x_j \in E^*$ into six categories. The first three categories are the most fundamental, which correspond to three main ways of melody reduction in Schenkerian analysis: prolongation, linear progression, and arpeggiation [21]. Note that the edges are strictly defined below, and we only borrow the terms for implication:

- **Prolongational Edge (PE)**: x_j prolongs x_i with the same pitch. For example, a PE can potentially remove a neighbor tone. Mathematically, a PE satisfies $\text{Pitch}(x_i) = \text{Pitch}(x_j)$ and $\text{Onset}(x_j) - \text{Onset}(x_i) < D$.
- **Linear Edge (LE)**: the interval between x_i and x_j is a second. For example, an LE can potentially mark a significant melodic movement. Mathematically, an LE satisfies $|\text{Pitch}(x_i) - \text{Pitch}(x_j)| \in \{1, 2\}$ and $\text{Onset}(x_j) - \text{Onset}(x_i) < D$.
- **Arpeggiation Edge (AE)**: the interval between x_i and x_j is larger than a (compound) second and x_i and x_j are within the same chord. For example, an AE can potentially mark an elaboration of harmony. Mathematically, an AE satisfies $|\text{PitchClass}(x_i) - \text{PitchClass}(x_j)| \in \{3, 4, 5, 6, 7, 8, 9\}$ and $\text{Chord}(x_i) = \text{Chord}(x_j)$.

In some melody compositions, pitches that span an octave are also regarded as a smooth connection. In Schenkerian analysis, this is explained by *imaginary continuo*—although the two tones span an octave in the current realization, they are close in other imaginary realizations. We define two types of imaginary edges accordingly:

- **Imaginary Prolongational Edge (IPE)**: x_j prolongs x_i with the same pitch class. Mathematically, an IPE (is not a PE) and satisfies $\text{PitchClass}(x_i) = \text{PitchClass}(x_j)$ and $\text{Onset}(x_j) - \text{Onset}(x_i) < D$.
- **Imaginary Linear Edge (ILE)**: the interval between x_i and x_j (or its inversion) is a compound second. Mathematically, an ILE (is not an LE) and satisfies $|\text{PitchClass}(x_i) - \text{PitchClass}(x_j)| \in \{1, 2, 10, 11\}$ and $\text{Onset}(x_j) - \text{Onset}(x_i) < D$.

Finally, all the rest of the edges in E^* belong to the final category. This is to ensure the graph is connected so that there must exist at least one path from x_1 to x_N:

- **Unclassified Edges (UE)**: the rest of the edges.

In the above definition, $\text{Pitch}(\cdot)$, $\text{Onset}(\cdot)$, and $\text{Chord}(\cdot)$ are previously defined in Section 3.1. $\text{PitchClass(x)} := \text{Pitch}(x) \mod 12$ and $\text{Chord}(x_i) = \text{Chord}(x_j)$ if and only if x_i and x_j are within the interval of a single chord. In our experiment, the temporal threshold D is set to 2 measures.

3.3 Edge Cost Definition

We define the edge cost function of $x_i \to x_j$ so that a more significant edge will have a smaller cost. The edge cost function considers three aspects: 1) the function of different edge types, 2) the temporal distance of an edge, and 3) the note importance. [2]

First, we define *tonal cost*, denoted by $c_{\text{tonal}}(x_i \to x_j)$, prioritizing prolongational and linear edges in the melody reduction. Formally,

$$c_{\text{tonal}}(x_i \to x_j) := \begin{cases} 0.1, & \text{if } x_i \to x_j \text{ is a PE,} \\ 0.3, & \text{if } x_i \to x_j \text{ is an LE,} \\ 1.5, & \text{if } x_i \to x_j \text{ is an AE,} \\ 1.0, & \text{if } x_i \to x_j \text{ is an IPE,} \\ 1.3, & \text{if } x_i \to x_j \text{ is an ILE,} \\ 3.0, & \text{if } x_i \to x_j \text{ is a UE.} \end{cases} \quad (1)$$

Then, we define the *temporal cost*, denoted by $c_{\text{temp}}(x_i \to x_j)$, to measure the distance from index i to index j. We set the hyperparameter $\eta = 1.6$ to achieve an ideal degree of reduction. A larger η results in too little reduction and a smaller η makes the reduction too coarse:

$$c_{\text{temp}}(x_i \to x_j) := (j - i)^{\eta}. \quad (2)$$

Besides the two cost functions on edges, we introduce a *note importance factor*, denoted by $\alpha(x_i)$, to ensure structurally important notes are more likely to be selected. Particularly, $\alpha(x_i)$ is a product of four terms:

$$\alpha(x) := \alpha_{\text{p}}(x)\alpha_{\text{o}}(x)\alpha_{\text{d}}(x)\alpha_{\text{h}}(x), \quad (3)$$

[2] Currently, the costs are empirically specified based on domain knowledge and preliminary analysis. Estimating them from data is left for future work.

where $\alpha_{\text{p}}(x_i)$ denotes *pitch importance*, $\alpha_{\text{o}}(x_i)$ denotes *onset importance*, $\alpha_{\text{d}}(x_i)$ denotes *duration importance*, and $\alpha_{\text{h}}(x_i)$ denotes *harmony importance*.

1. **Pitch Importance.** Higher and lower pitches are usually more significant in a melody and should be given a smaller weight factor:

$$\alpha_{\text{p}}(x_i) := 0.1 \times \left(0.5 - \frac{|\text{Pitch}(x_i) - p_{\text{mid}}|}{p_{\text{max}} - p_{\text{mid}}}\right) + 1, \quad (4)$$

where p_{max} and p_{min} are maximum and minimum pitch values and $p_{\text{mid}} = (p_{\text{max}} + p_{\text{min}})/2$.

2. **Onset Importance.** The notes having higher metrical importance should be given a smaller weight factor:

$$\alpha_{\text{o}}(x_i) := \begin{cases} 0.85, & \text{Onset}(x_i) \in \text{DB}, \\ 0.95, & \text{Onset}(x_i) \in \text{B}, \\ 1.05, & \text{Onset}(x_i) \in \text{B/2}, \\ 1.15, & \text{Onset}(x_i) \in \text{B/4}. \end{cases} \quad (5)$$

Here, DB, B, B/2, and B/4 represent downbeat, beat, eighth-note, and sixteenth-note positions, respectively (if under the 4/4 time signature).

3. **Duration Importance.** Longer notes should be given a smaller weight factor:

$$\alpha_{\text{d}}(x_i) := \begin{cases} 0.85, & \text{Dur}(x_i) \geq \text{half note}, \\ 0.95, & \text{Dur}(x_i) \geq \text{quarter note}, \\ 1.05, & \text{Dur}(x_i) \geq 8^{\text{th}} \text{ note}, \\ 1.15, & \text{Dur}(x_i) \geq 16^{\text{th}} \text{ note}. \end{cases} \quad (6)$$

4. **Harmony Importance**. A chord tone should be given a smaller weight factor than non-chord tones.

$$\alpha_{\text{h}}(x_i) := \begin{cases} 0.85, & x_i \text{ is a chord tone}, \\ 1.15, & \text{otherwise}. \end{cases} \quad (7)$$

Here x_i is a chord tone strictly means $\text{PitchClass}(x_i)$ is in $\text{Chord}(x_i)$. So, an anticipation is regarded as a chord tone to the next chord (see Section 3.1).

Finally, the total edge cost is defined as a summation of tonal and temporal cost, modulated by the note importance factor:

$$c(x_i \to x_j) = \alpha(x_j)[c_{\text{temp}}(x_i \to x_j) + c_{\text{tonal}}(x_i \to x_j)]. \quad (8)$$

Thus, the melody reduction can be achieved by running a shortest-path algorithm to find the shortest path from x_1 to x_N.

3.4 Post-Processing

After we find the shortest path, we use a rule-based post-processing method to arrange the selected notes in the path to melody reduction. The maximum resolution of the reduction is a quarter note, in the style of a fifth-species counterpoint [22].

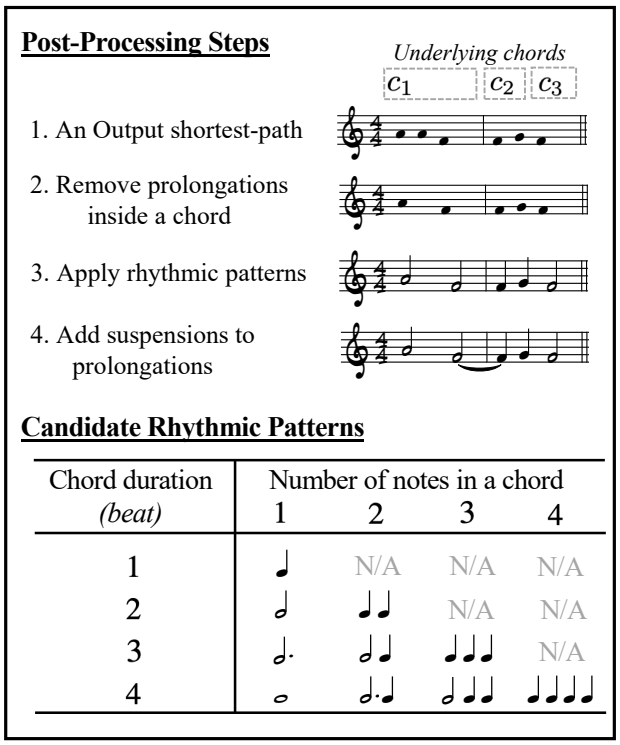


Figure 2: An illustration of post-processing operations.

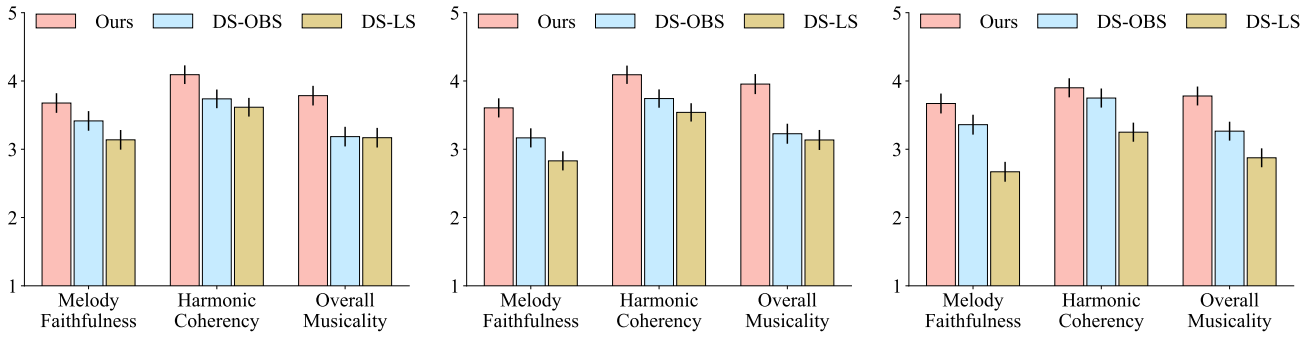
Figure 2 shows the detailed procedure. First, the nodes in the shortest path are allocated into *chord bins*, with each bin corresponding to a distinct chord. In each chord bin, the notes are given a fixed rhythm template (see the table at the bottom of Figure 2), ensuring the notes within a bin collectively span the entire duration of their associated chord. In this process, notes linked by a prolongational edge are merged into a single note. If the number of nodes in a chord bin exceeds the length of the chord, a random selection of notes will be omitted. Finally, the prolongational edges between two chords are marked with suspension. Note that notes serving as anticipations are allocated to the bin of the subsequent chord.

4. EXPERIMENTS

In Section 4.1, we evaluate the proposed algorithm through a subjective listening test. In Section 4.2, we show and evaluate a melody reduction example as a case study. Finally, we evaluate the effectiveness of melody reduction in downstream music generation tasks in Section 4.3.

4.1 Subjective Evaluation of Melody Reduction

Unlike tasks with clear ground truths, melody reduction is inherently subjective and style-dependent. Existing theories, such as GTTM or Schenkerian analysis, provide interpretive hierarchies rather than prescriptive outcomes [6, 21]. Finding reduction typically involves pruning a tree at variable depths, often informed by human judgment. Moreover, such theories are primarily suited to classical music.



(a) Pop Genre (b) Folk Genre (c) Classical Genre

Figure 3: Subjective evaluation results of melody reduction quality across three genres.

Given these challenges, we adopt a subjective listening test to better capture the perceptual and musical quality of melody reductions. We cover three music genres: pop, folk, and classical. We sample melodies from the POP909 dataset [23], the Nottingham dataset [24], and the GTTM database [25] for the pop, folk, and classical genres, respectively. We compare with two representative baselines commonly used for melody reduction as feature extraction in music generation:

- **Downsampling on Observations** (*DS-OBS*): From a statistical perspective, the melody is downsampled to a sequence of half notes, each representing the most common pitch in the 2-beat music segment [1].
- **Downsampling in Latent Space** (*DS-LS*): EC^2-VAE [26] learns disentangled latent representations of the pitch contour and rhythmic pattern of 2-measure music segments as z_{p} and z_{r}, respectively, which enables downsampling in the latent space of rhythm patterns. Specifically, we encode the pitch contour z_{p} of data and decode it together with a downsampled rhythm z_{r} to get the melody reduction for every 2-measure segment.

For the subjective test, we randomly select four 8-measure melodies from each genre. Each participant listens to at least 3 groups of melody reductions for each genre. In each group, participants are presented with the original melody first, followed by the melody reductions generated by the proposed algorithm, downsampling, and latent representation recombination in a randomized order. Participants are asked to rate the quality of the melody reduction on a 5-point Likert scale, where 1 indicates the worst quality and 5 indicates the best, in terms of three criteria: (1) *Melody Faithfulness*: how well the melody reduction preserves the original music information. (2) *Harmonic Coherency*: how well the melody reduction fits the underlying chord progression. (3) *Overall Musicality*: the overall music quality of the melody reduction.

A total of 45 subjects (26 females & 19 males) participated in the survey, in which over 70% have a music education experience of at least 2 years. The results are reported in Figure 3, where the heights of bars represent means of the ratings and the error bars represent the standard error computed by within-subject ANOVA [27]. The proposed algorithm is significantly preferred over both baselines in all genres and criteria ($p < 0.05$), except for melody faithfulness in the pop genre, where the difference shows a positive but marginal trend ($p < 0.075$).

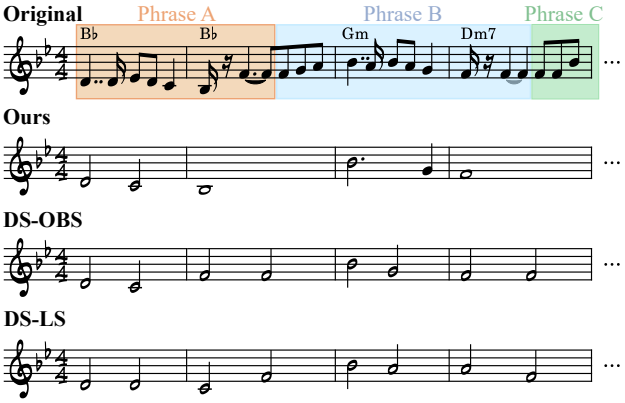


Figure 4: Comparison of the original melody, melody reductions from the proposed method, and the baselines. We highlight the phrases in the top row.

4.2 A Case Study of Melody Reduction

We provide a case analysis of melody reduction comparing the proposed algorithm and baselines introduced in Section 4.1, as shown in Figure 4. The original melody is shown in the top row, followed by the three melody reductions generated by the proposed algorithm and baselines.

Both the proposed method and DS-OBS can mostly capture the correct melody flow, such as important passing tones like C4 in the first measure. In subtle situations such as the second measure, where Phrase A ends its downward music flow with the downbeat chord tone B♭3 and lingers at F4 until the transition to Phrase B, DS-OBS fails to preserve B♭3, as it is overshadowed by the long duration of F4. In contrast, the proposed algorithm successfully captures B♭3 by paying attention to its harmonic and rhythmic importance, as well as the imaginary prolongational edge between B♭3 and B♭4 in the third measure. DS-LS only captures the pitch contour but introduces several unwanted non-chord tones. This example demonstrates the effectiveness of the proposed algorithm for melody reduction.

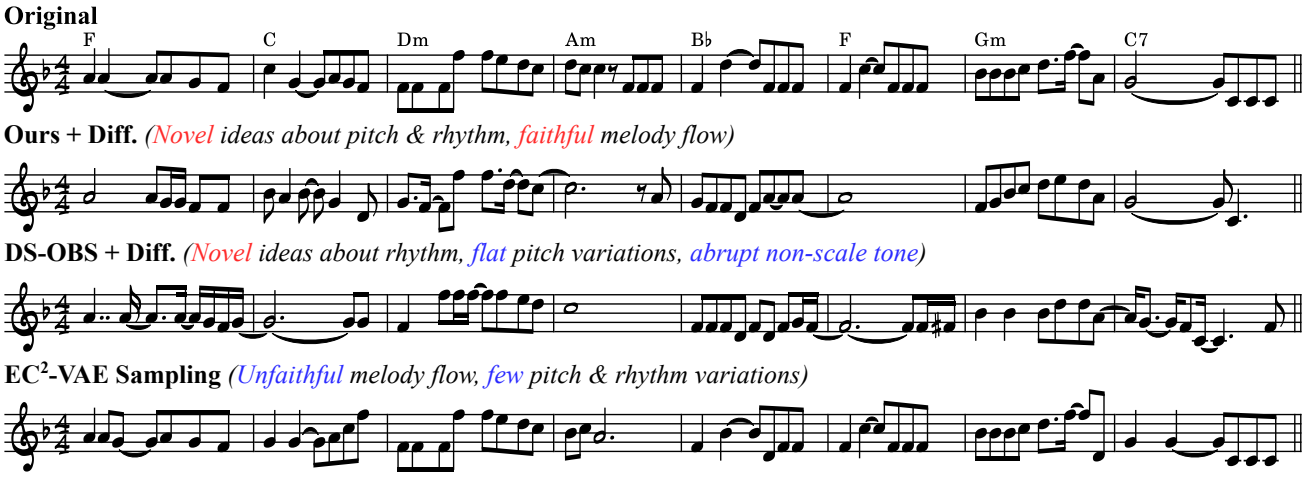


Figure 5: Comparison of variations of an example melody. Here **Diff.** denotes the conditional diffusion model trained to generate melody variations from melody reductions. Positive comments are highlighted in red, negatives in blue.

4.3 Downstream Task: Generating Melody Variations

We believe melody reduction can serve as a useful representation of structural information in downstream tasks. In this section, we demonstrate one such application in a melody variation generation task. The task uses the reduction of a melody as input, and outputs variations faithful to the original melody. While we do not claim a strong causal link between reduction quality and generation quality, our intuition is that an accurate reduction better reflects the underlying melodic and harmonic context, which in turn supports more coherent and musically grounded generation.

To this end, we train a diffusion model to generate melody variations from melody reductions provided by the proposed algorithm. We use a similar model design and training settings as the leadsheet generation model in [3] and train the model on the POP909 dataset. Similarly, we train the model using melody reductions by DS-OBS. We also generate melody variations by sampling in the latent space of z_{p} and z_{r} of EC^2-VAE for comparison. For all methods, we randomly sample four outputs per input and select the most representative one for use in listening tests.

Figure 5 shows a group of melody variation examples. It can be seen that the variation model trained with the proposed melody reductions not only maintains the original melody flow but also introduces novel ideas in pitch and rhythm. The model trained with DS-OBS also preserves the pitch contour, but tends to have flat pitch variations. The variation generated by sampling from the latent space of EC^2-VAE changes the original melody flow in an unwanted way, and does not introduce rich variations.

We evaluate the melody variations on the test set of POP909 using a subjective listening test with the same participants as in Section 4.1. Each participant listens to at least three groups of melody variations, where participants are first presented with the original melody, followed by the three variations in random order. Participants are asked to rate the quality of melody variations and the original melody by human composers in three criteria: *Naturalness*, *Creativity*, and *Musicality* [28]. The results are reported in Figure 6, with the same computation as in Section 4.1, including mean ratings and statistical significance tests. Our method is consistently preferred over two baselines in terms of creativity and overall musicality ($p < 0.05$), and remains competitive in naturalness.

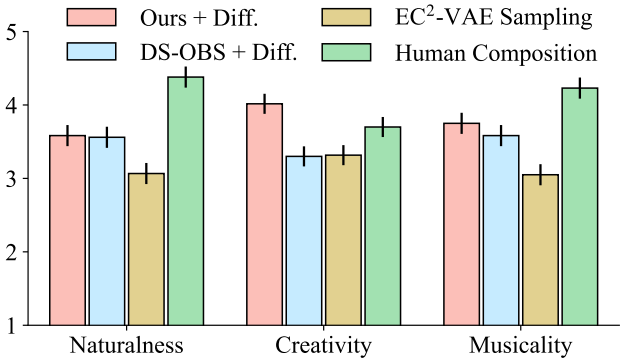


Figure 6: Subjective results of melody variations.

5. CONCLUSION

To sum up, this work proposed a novel and useful algorithm for melody reduction, filling the gap between the need to capture melody flow for long-term and hierarchical music generation and the lack of all-genre off-the-shelf tools for melody reduction. The proposed algorithm finds the optimal melody reduction by finding the shortest path in a graph representation of the melody, considering the tonal, temporal, and note importance factors. Subjective experiments demonstrated that our method outperforms baselines in a variety of musical styles. We also demonstrated the effectiveness of the melody reduction algorithm in melody variation generation through subjective evaluation. In the future, we plan to tackle reduction that captures latent polyphony and hierarchical structure, and explore the application of the proposed algorithm in a broader range of music generation tasks. While the current algorithm contains ad-hoc parameters, future work could also explore learning these directly from data.

6. REFERENCES

[1] S. Dai, Z. Jin, C. Gomes, and R. B. Dannenberg, "Controllable deep melody generation via hierarchical music structure representation," in *Proceedings of the 22nd International Society for Music Information Retrieval Conference, ISMIR 2021, Online, November 7-12, 2021*, J. H. Lee, A. Lerch, Z. Duan, J. Nam, P. Rao, P. van Kranenburg, and A. Srinivasamurthy, Eds., 2021, pp. 143–150. [Online]. Available: https://archives.ismir.net/ismir2021/paper/000017.pdf

[2] S. Wei, G. Xia, Y. Zhang, L. Lin, and W. Gao, "Music phrase inpainting using long-term representation and contrastive loss," in *IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2022, Virtual and Singapore, 23-27 May 2022*. IEEE, 2022, pp. 186–190. [Online]. Available: https://doi.org/10.1109/ICASSP43922.2022.9747817

[3] Z. Wang, L. Min, and G. Xia, "Whole-song hierarchical generation of symbolic music using cascaded diffusion models," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=sn7CYWyavh

[4] P. Dhariwal, H. Jun, C. Payne, J. W. Kim, A. Radford, and I. Sutskever, "Jukebox: A generative model for music," *CoRR*, vol. abs/2005.00341, 2020. [Online]. Available: https://arxiv.org/abs/2005.00341

[5] H. Schenker, *Free Composition (Der freie Satz)*. New York: Longman, 1979, translated and edited by Ernst Oster.

[6] F. Lerdahl and R. S. Jackendoff, *A Generative Theory of Tonal Music, reissue, with a new preface*. MIT press, 1996.

[7] N. Orio and A. Rodà, "A measure of melodic similarity based on a graph representation of the music structure," in *Proceedings of the 10th International Society for Music Information Retrieval Conference, ISMIR 2009, Kobe International Conference Center, Kobe, Japan, October 26-30, 2009*, K. Hirata, G. Tzanetakis, and K. Yoshii, Eds. International Society for Music Information Retrieval, 2009, pp. 543–548. [Online]. Available: http://ismir2009.ismir.net/proceedings/OS7-1.pdf

[8] F. Simonetta, F. Carnovalini, N. Orio, and A. Rodà, "Symbolic music similarity through a graph-based representation," in *Proceedings of the Audio Mostly 2018 on Sound in Immersion and Emotion, Wrexham, United Kingdom, September 12-14, 2018*, S. Cunningham and R. Picking, Eds. ACM, 2018, pp. 26:1–26:7. [Online]. Available: https://doi.org/10.1145/3243274.3243301

[9] S. Tojo, K. Hirata, and M. Hamanaka, "Computational reconstruction of cognitive music theory," *New Gener. Comput.*, vol. 31, no. 2, pp. 89–113, 2013. [Online]. Available: https://doi.org/10.1007/s00354-013-0202-7

[10] R. Groves, "Automatic melodic reduction using a supervised probabilistic context-free grammar," in *Proceedings of the 17th International Society for Music Information Retrieval Conference, ISMIR 2016, New York City, United States, August 7-11, 2016*, M. I. Mandel, J. Devaney, D. Turnbull, and G. Tzanetakis, Eds., 2016, pp. 775–781. [Online]. Available: https://wp.nyu.edu/ismir2016/wp-content/uploads/sites/2294/2016/07/274_Paper.pdf

[11] S. Ni-Hahn, W. Xu, Z. Yin, R. Zhu, S. Mak, Y. Jiang, and C. Rudin, "A new dataset, notation software, and representation for computational schenkerian analysis," in *Proceedings of the 25th International Society for Music Information Retrieval Conference, ISMIR 2024, San Francisco, California, USA and Online, November 10-14, 2024*, B. Kaneshiro, G. J. Mysore, O. Nieto, C. Donahue, C. A. Huang, J. H. Lee, B. McFee, and M. C. McCallum, Eds., 2024, pp. 866–873. [Online]. Available: https://doi.org/10.5281/zenodo.14877467

[12] S. Ahlbäck, "Melody beyond notes: A study of melody cognition," Ph.D. dissertation, Göteborgs universitet, 2004.

[13] J. Y. Yen, "Finding the k shortest loopless paths in a network," *management Science*, vol. 17, no. 11, pp. 712–716, 1971.

[14] E. Narmour, *The analysis and cognition of basic melodic structures: The implication-realization model.* University of Chicago Press, 1990.

[15] P. B. Kirlin and P. E. Utgoff, "A framework for automated schenkerian analysis," in *ISMIR 2008, 9th International Conference on Music Information Retrieval, Drexel University, Philadelphia, PA, USA, September 14-18, 2008*, J. P. Bello, E. Chew, and D. Turnbull, Eds., 2008, pp. 363–368. [Online]. Available: http://ismir2008.ismir.net/papers/ISMIR2008_229.pdf

[16] M. Hamanaka, K. Hirata, and S. Tojo, "σGTTM III: Learning-based time-span tree generator based on pcfg," in *International Symposium on Computer Music Multidisciplinary Research*. Springer, 2015, pp. 387–404.

[17] ——, "deepGTTM-II: Automatic generation of metrical structure based on deep learning technique," in *13th Sound and Music Conference*, 2016, pp. 221–249.

[18] C. Finkensiep and M. Rohrmeier, "Modeling and inferring proto-voice structure in free polyphony," in *Proceedings of the 22nd International Society for Music Information Retrieval Conference, ISMIR 2021, Online, November 7-12, 2021*, J. H. Lee, A. Lerch, Z. Duan, J. Nam, P. Rao, P. van Kranenburg, and A. Srinivasamurthy, Eds., 2021, pp. 189–196. [Online]. Available: https://archives.ismir.net/ismir2021/paper/000023.pdf

[19] K. Chen, C. Wang, T. Berg-Kirkpatrick, and S. Dubnov, "Music sketchnet: Controllable music generation via factorized representations of pitch and rhythm," in *Proceedings of the 21th International Society for Music Information Retrieval Conference, ISMIR 2020, Montreal, Canada, October 11-16, 2020*, J. Cumming, J. H. Lee, B. McFee, M. Schedl, J. Devaney, C. McKay, E. Zangerle, and T. de Reuse, Eds., 2020, pp. 77–84. [Online]. Available: http://archives.ismir.net/ismir2020/paper/000146.pdf

[20] D. von Rütte, L. Biggio, Y. Kilcher, and T. Hofmann, "FIGARO: controllable music generation using learned and expert features," in *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023. [Online]. Available: https://openreview.net/pdf?id=NyR8OZFHw6i

[21] A. C. Cadwallader, D. Gagné, and F. Samarotto, "Analysis of tonal music: a schenkerian approach," *(No Title)*, 1998.

[22] M. Clementi, C. Tausig, and K. F. Weitzmann, *Gradus ad parnassum*. Peters, 2010.

[23] Z. Wang, K. Chen, J. Jiang, Y. Zhang, M. Xu, S. Dai, G. Bin, and G. Xia, "Pop909: A pop-song dataset for music arrangement generation," in *Proceedings of 21st International Conference on Music Information Retrieval, ISMIR*, 2020.

[24] E. Foxley, "Nottingham database," 2011.

[25] M. Hamanaka, "Gttm database," https://gttm.jp/gttm/database/, 2009.

[26] R. Yang, D. Wang, Z. Wang, T. Chen, J. Jiang, and G. Xia, "Deep music analogy via latent representation disentanglement," in *Proceedings of the 20th International Society for Music Information Retrieval Conference, ISMIR 2019, Delft, The Netherlands, November 4-8, 2019*, A. Flexer, G. Peeters, J. Urbano, and A. Volk, Eds., 2019, pp. 596–603. [Online]. Available: http://archives.ismir.net/ismir2019/paper/000072.pdf

[27] H. Scheffe, *The analysis of variance*. John Wiley & Sons, 1999, vol. 72.

[28] H. Chu, J. Kim, S. Kim, H. Lim, H. Lee, S. Jin, J. Lee, T. Kim, and S. Ko, "An empirical study on how people perceive ai-generated music," in *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, 2022, pp. 304–314.